

Day 7 Beginner-Friendly Full Slide Content

Course metadata

Course Title: G1 AI-Driven Locomotion, Audio, and Application Assembly **Day:** 7 **Target Audience:** Beginners **Instructor:** Manus AI

Teaching design notes

This deck is designed for beginners, focusing on the practical integration of AI locomotion, audio, and speech interfaces into a unified G1 application. It simplifies complex AI and multimodal concepts, emphasizing hands-on understanding of interfacing, synchronization, and assembly. Each slide includes clear purpose statements, on-slide text, detailed speaker notes, and cues for visual diagrams to enhance learning.

Section summary table

Section	Focus	Suggested Slide Range
Cover and Learning Map	Day 7 scope, multimodal flow, the “unified robot”	1–4
AI Locomotion & Benchmarks	AI engines, data interfacing, state validation	5–16
Audio & Speech Infrastructure	Audio setup, ASR, TTS, WAVs, and LED sync	17–30
Unified Application Assembly	Blending motion/voice/manipulation, packaging	31–45
Final Validation & Deployment	Full-stack testing, deployment habits, course wrap-up	46–56

Cover

Section: Cover **Purpose:** Introduce the course day and its focus. **On-slide text:**

Day 7: G1 AI-Driven Locomotion, Audio, and Application Assembly

Speaker notes: Welcome to Day 7! Today marks the culmination of our G1 journey as we integrate advanced AI locomotion, audio capabilities, and speech interfaces into a fully unified application. We'll learn how to bring all these complex systems together to create a truly interactive and intelligent humanoid robot.

Diagram cue: Vinci AI logo, subtle diagonal watermark, copyright tag.

Slide 1: Learning Map & The Unified Robot

Section: Cover and Learning Map **Purpose:** Outline the day's learning objectives and introduce the concept of a unified robot. **On-slide text:**

Learning Map: Assembling the Intelligent G1

Initialize AI Engine → Validate State → Setup Audio → Integrate Speech → Sync Multimodal → Deploy App

The Unified Robot: More Than the Sum of Its Parts

Speaker notes: Our path today will guide us through interfacing with AI locomotion engines, setting up G1's audio, integrating speech capabilities, and finally, synchronizing all these elements into a cohesive application. The goal is to create a unified robot that seamlessly blends motion, voice, and manipulation.

Diagram cue: Flowchart with arrows illustrating the learning path. A diagram of a G1 humanoid with interconnected modules representing different functionalities.

Slide 2: AI-Driven Locomotion Engines - Introduction

Section: AI Locomotion & Benchmarks **Purpose:** Introduce the concept of AI-driven locomotion engines in G1. **On-slide text:**

Smart Steps: AI-Driven Locomotion Engines

- **Definition:** Software modules that use AI (e.g., reinforcement learning, optimization) to generate dynamic gaits.
- **Benefits:** More adaptive, robust, and natural-looking movements.
- **Integration:** How G1 interfaces with these advanced control systems.

Speaker notes: G1 can leverage AI-driven locomotion engines to achieve highly adaptive and natural movements. These engines use sophisticated algorithms to generate gaits that can respond to changing environments and tasks. We'll explore how G1 integrates with these powerful AI systems.

Diagram cue: Diagram showing an AI locomotion engine block connected to G1's motor control system.

Slide 3: Interfacing with AI-Driven Locomotion Engines

Section: AI Locomotion & Benchmarks **Purpose:** Explain the practical aspects of connecting G1 to AI locomotion engines. **On-slide text:**

Connecting Minds: Interfacing with AI Engines

- **Standard APIs:** Common interfaces for sending commands and receiving feedback.
- **Data Formats:** JSON or Protobuf for structured data exchange.
- **Real-time Requirements:** Low-latency communication for dynamic control.

Speaker notes: Interfacing with AI locomotion engines involves using standard APIs and data formats to ensure seamless communication. Real-time performance is critical, as G1 needs to respond instantly to the AI's commands to maintain balance and execute movements smoothly. We'll discuss the practicalities of this connection.

Diagram cue: Data flow diagram showing G1's control system exchanging data with an external AI locomotion engine via an API.

Slide 4: Data Flow Between AI Models and Physical Controllers

Section: AI Locomotion & Benchmarks **Purpose:** Illustrate the flow of data from AI models to G1's physical actuators. **On-slide text:**

The Loop: AI Model to Physical Controller Data Flow

- **AI Output:** Desired joint trajectories, forces, or high-level actions.
- **Trajectory Generation:** Converts AI output into motor commands.
- **Motor Control:** Executes commands on physical actuators.
- **Sensor Feedback:** Provides real-world data back to the AI for learning/adaptation.

Speaker notes: The data flow from an AI model to G1's physical controllers is a continuous loop. The AI generates desired movements, which are then translated into specific motor commands. Sensor feedback from G1's body is crucial, as it allows the AI to learn, adapt, and refine its control strategies in real-time.

Diagram cue: Detailed data flow diagram showing AI model -> Trajectory Generation -> Motor Control -> G1 Actuators, with Sensor Feedback looping back to the AI model.

Slide 5: State Validation Benchmarks - Introduction

Section: AI Locomotion & Benchmarks **Purpose:** Introduce the concept of state validation benchmarks for AI-driven locomotion. **On-slide text:**

Ensuring Safety: State Validation Benchmarks

- **Definition:** Metrics and tests to verify G1's state (position, velocity, balance) against safe operating parameters.
- **Importance:** Prevents unsafe movements and ensures predictable behavior.
- **Types:** Static posture checks, dynamic stability tests, trajectory adherence.

Speaker notes: State validation benchmarks are essential for ensuring the safe and accurate operation of AI-driven locomotion. These benchmarks define acceptable ranges for G1's physical state and help us detect any deviations that could lead to instability or damage. We'll look at different types of validation tests.

Diagram cue: A dashboard-like graphic showing various G1 state parameters with green/red indicators for validation status.

Slide 6: Implementing Static Posture Checks

Section: AI Locomotion & Benchmarks **Purpose:** Explain how to implement static checks for G1's posture. **On-slide text:**

Standing Still: Static Posture Validation

- **Joint Angle Limits:** Verify all joints are within safe angular ranges.
- **Center of Mass (CoM) Projection:** Ensure CoM is within the support polygon.
- **Ground Contact:** Confirm expected foot-ground contact points.

Speaker notes: Static posture checks are fundamental validation benchmarks. They involve verifying that G1's joints are within their safe limits, its Center of Mass is correctly projected within its support base, and all expected contact points with the ground are maintained. These checks are crucial before any movement begins.

Diagram cue: Diagram of a G1 humanoid in a static pose, with annotations for joint limits, CoM projection, and ground contact points.

Slide 7: Dynamic Stability Tests

Section: AI Locomotion & Benchmarks **Purpose:** Describe methods for testing G1's stability during movement. **On-slide text:**

Moving Safely: Dynamic Stability Validation

- **Zero Moment Point (ZMP) Tracking:** Monitor ZMP trajectory during walking.
- **External Perturbation Response:** Test G1's ability to recover from pushes or disturbances.
- **Fall Detection & Recovery:** Verify proper response to loss of balance.

Speaker notes: Dynamic stability tests assess G1's ability to maintain balance while moving. This includes monitoring the Zero Moment Point, testing its response to external pushes, and verifying its fall detection and recovery mechanisms. These benchmarks are vital for robust locomotion.

Diagram cue: Graph showing ZMP trajectory within the support polygon during a walking cycle. A G1 humanoid recovering from a slight push.

Slide 8: Trajectory Adherence Benchmarks

Section: AI Locomotion & Benchmarks **Purpose:** Explain how to measure G1's ability to follow commanded trajectories. **On-slide text:**

Following the Path: Trajectory Adherence Validation

- **Position Error:** Difference between commanded and actual end-effector position.
- **Orientation Error:** Difference in commanded and actual end-effector orientation.
- **Path Deviation:** How closely G1 follows a planned path.

Speaker notes: Trajectory adherence benchmarks measure how accurately G1 follows its commanded movements. This involves quantifying position and orientation errors of its end-effectors (like hands or feet) and assessing how much it deviates from a planned path. High adherence is crucial for precision tasks.

Diagram cue: Graph showing commanded vs. actual end-effector trajectory, highlighting position and orientation errors.

Slide 9: G1 Audio Infrastructure - Introduction

Section: Audio & Speech Infrastructure **Purpose:** Introduce G1's audio capabilities and infrastructure. **On-slide text:**

G1's Voice and Ears: Audio Infrastructure

- **Purpose:** Enable G1 to produce sounds (speech, alerts) and perceive audio (speech recognition).
- **Components:** Microphones, speakers, audio processing units.
- **Integration:** How audio systems connect to G1's central control.

Speaker notes: G1's audio infrastructure allows it to interact with the world through sound. This includes both generating audio, such as speaking or playing alerts, and perceiving audio through microphones for speech recognition. We'll explore the components and how they integrate into G1's overall system.

Diagram cue: Diagram of a G1 humanoid with microphones and speakers highlighted, showing audio input/output pathways.

Slide 10: Setting Up Audio Parameter Classes

Section: Audio & Speech Infrastructure **Purpose:** Explain how to define audio parameter classes for G1. **On-slide text:**

Defining Sound: Audio Parameter Classes

- **Volume Control:** `AudioParams.volume = 0.7`.
- **Playback Device:** `AudioParams.output_device = 'speaker_1'`.
- **Sample Rate/Format:** Ensure compatibility with audio files.

Speaker notes: To manage audio effectively, we use audio parameter classes. These classes define settings like volume, the specific output device to use, and technical details like sample rate and format. This structured approach ensures consistent audio playback across different applications.

Diagram cue: Code snippet showing the definition of an `AudioParams` class with volume and device properties.

Slide 11: Defining Service Constants for Audio Handling

Section: Audio & Speech Infrastructure **Purpose:** Discuss the use of service constants for managing audio services. **On-slide text:**

Standardizing Audio: Service Constants

- **Service Names:** `AUDIO_PLAY_SERVICE = '/g1/audio/play'`.
- **Status Codes:** `AUDIO_SUCCESS = 0`, `AUDIO_ERROR = -1`.
- **Benefits:** Consistency, easier debugging, and maintainability.

Speaker notes: Service constants are predefined values used to interact with G1's audio services. They define standard names for services, like playing audio, and status codes for success or error. Using constants makes our code more consistent, easier to read, and simpler to debug.

Diagram cue: A table listing common audio service constants and their corresponding values.

Slide 12: Implementing Audio Playback Routines

Section: Audio & Speech Infrastructure **Purpose:** Explain how to implement routines for playing audio files on G1. **On-slide text:**

Making G1 Speak: Audio Playback Routines

- **Load File:** Read WAV or MP3 file into memory.
-

- **Apply Parameters:** Set volume and output device using `AudioParams`.
- **Call Service:** Send playback request to the audio service.
- **Handle Response:** Check status codes for success or failure.

Speaker notes: Implementing audio playback involves a clear routine: loading the audio file, applying our predefined parameters, calling the appropriate audio service, and handling the response. We'll walk through this process to ensure G1 can reliably play sounds and speech.

Diagram cue: Flowchart showing the steps for audio playback: Load File -> Apply Params -> Call Service -> Handle Response.

Slide 13: Multimodal Speech Interfaces - Introduction

Section: Audio & Speech Infrastructure **Purpose:** Introduce the concept of multimodal speech interfaces. **On-slide text:**

Beyond Sound: Multimodal Speech Interfaces

- **Definition:** Combining speech with other modalities (e.g., vision, motion) for richer interaction.
- **Goal:** Create more natural and intuitive human-robot communication.
- **Components:** ASR, TTS, visual feedback (LEDs), physical gestures.

Speaker notes: Multimodal speech interfaces go beyond just sound. They combine speech recognition and generation with visual feedback, like LED arrays, and physical gestures to create a more natural and engaging interaction experience. We'll explore how to integrate these different modalities.

Diagram cue: Diagram showing a user speaking to G1, with G1 processing the speech, responding with audio, and displaying visual feedback on its LEDs.

Slide 14: Integrating Automatic Speech Recognition (ASR)

Section: Audio & Speech Infrastructure **Purpose:** Explain how to integrate ASR for G1 to understand spoken commands. **On-slide text:**

Listening to You: Integrating ASR

- **ASR Engine:** Converts spoken audio into text.
- **Microphone Input:** Capturing clear audio for processing.

- **Intent Parsing:** Extracting meaning from the transcribed text.

Speaker notes: Automatic Speech Recognition (ASR) is how G1 “hears” and understands us. We’ll discuss integrating an ASR engine, ensuring clear microphone input, and the crucial step of intent parsing—translating the transcribed text into actionable commands for G1.

Diagram cue: Flowchart showing Microphone Input -> ASR Engine -> Transcribed Text -> Intent Parsing -> Actionable Command.

Slide 15: Integrating Text-to-Speech (TTS)

Section: Audio & Speech Infrastructure **Purpose:** Explain how to integrate TTS for G1 to generate spoken responses. **On-slide text:**

G1’s Voice: Integrating TTS

- **TTS Engine:** Converts text into spoken audio.
- **Voice Selection:** Choosing an appropriate voice profile for G1.
- **Dynamic Responses:** Generating speech based on G1’s state or actions.

Speaker notes: Text-to-Speech (TTS) gives G1 its voice. We’ll cover integrating a TTS engine, selecting a suitable voice profile, and how to generate dynamic spoken responses based on G1’s current state or the actions it’s performing, making interactions more conversational.

Diagram cue: Flowchart showing Text Input -> TTS Engine -> Audio Output -> G1 Speaker.

Slide 16: Managing WAV File Playback

Section: Audio & Speech Infrastructure **Purpose:** Discuss the management of pre-recorded audio files (WAVs). **On-slide text:**

Pre-recorded Sounds: Managing WAV Files

- **Use Cases:** Alerts, sound effects, standard greetings.
- **Storage & Retrieval:** Organizing audio assets efficiently.
- **Playback Integration:** Using the audio playback routines we defined earlier.

Speaker notes: While TTS is great for dynamic speech, pre-recorded WAV files are essential for specific sound effects, alerts, or standard greetings. We’ll discuss how to manage these

audio assets efficiently and integrate their playback into our applications using the routines we established.

Diagram cue: A folder structure showing organized WAV files (e.g., `/audio/alerts/` , `/audio/greetings/`).

Slide 17: Coordinating Audio with LED Arrays

Section: Audio & Speech Infrastructure **Purpose:** Explain how to synchronize audio playback with visual feedback using LEDs. **On-slide text:**

Visual Voice: Coordinating Audio and LEDs

- **Purpose:** Provide visual cues during speech or audio playback.
- **Synchronization:** Matching LED patterns to audio intensity or events.
- **Implementation:** Sending simultaneous commands to audio and LED services.

Speaker notes: To enhance the multimodal experience, we can coordinate audio playback with G1's LED arrays. This provides visual cues, like LEDs pulsing when G1 speaks, making the interaction more engaging. We'll learn how to synchronize these two modalities effectively.

Diagram cue: Diagram showing simultaneous commands being sent to the Audio Service and the LED Control Service, resulting in synchronized output.

Slide 18: Complete Application Assembly - Introduction

Section: Unified Application Assembly **Purpose:** Introduce the concept of assembling a complete, unified application. **On-slide text:**

The Final Piece: Complete Application Assembly

- **Goal:** Blend voice control, bipedal motion, and object manipulation.
- **Challenge:** Synchronizing multiple complex subsystems.
- **Outcome:** A unified, interactive, and capable G1 application.

Speaker notes: Now we reach the final piece: assembling a complete application. This involves blending everything we've learned—voice control, bipedal motion, and object manipulation—into a single, unified package. The challenge lies in synchronization, but the outcome is a truly capable robot.

Diagram cue: A puzzle graphic with pieces representing Voice, Motion, and Manipulation coming together to form a complete G1 application.

Slide 19: Blending Voice Control and Bipedal Motion

Section: Unified Application Assembly **Purpose:** Discuss the integration of voice commands with locomotive actions. **On-slide text:**

“Walk Forward”: Blending Voice and Motion

- **Voice Command:** User says “Walk forward.”
- **ASR & Intent:** System recognizes command and intent.
- **Locomotion Trigger:** High-level controller sends velocity command to FSM.
- **Feedback:** G1 confirms action via TTS (“Walking forward”).

Speaker notes: Let’s look at blending voice and motion. When a user says “Walk forward,” the ASR system processes it, extracts the intent, and triggers the locomotion controller. G1 then executes the movement and provides verbal confirmation via TTS. This is a seamless multimodal interaction.

Diagram cue: Flowchart showing Voice Command -> ASR -> Intent Parsing -> Locomotion Controller -> G1 Movement, with a parallel path for TTS confirmation.

Slide 20: Blending Voice Control and Object Manipulation

Section: Unified Application Assembly **Purpose:** Discuss the integration of voice commands with arm manipulation tasks. **On-slide text:**

“Grab That”: Blending Voice and Manipulation

- **Voice Command:** User says “Grab the red cup.”
- **ASR & Intent:** System recognizes command and target object.
- **Arm Control Trigger:** High-level API initiates grasping sequence.
- **Feedback:** G1 confirms action via TTS (“Grabbing the red cup”).

Speaker notes: Similarly, we can blend voice control with object manipulation. A command like “Grab the red cup” is processed, the target is identified, and the arm control API is triggered to execute the grasp. Again, TTS provides confirmation, creating a natural interaction loop.

Diagram cue: Flowchart showing Voice Command -> ASR -> Intent Parsing -> Arm Control API -> G1 Grasping Action, with a parallel path for TTS confirmation.

Slide 21: Synchronizing Motion, Voice, and Manipulation

Section: Unified Application Assembly **Purpose:** Explain the complexities of synchronizing all three modalities. **On-slide text:**

The Symphony: Synchronizing All Modalities

- **Concurrency:** Managing multiple tasks simultaneously.
- **State Management:** Ensuring actions don't conflict (e.g., walking while grasping).
- **Event-Driven Architecture:** Using events to trigger coordinated actions across subsystems.

Speaker notes: The true symphony happens when we synchronize motion, voice, and manipulation. This requires managing concurrent tasks, careful state management to prevent conflicting actions, and an event-driven architecture to ensure all subsystems work together harmoniously.

Diagram cue: A complex flowchart showing an event-driven architecture coordinating Locomotion, Arm Control, and Audio/Speech subsystems.

Slide 22: Practical Workflows for Packaging

Section: Unified Application Assembly **Purpose:** Discuss how to package a complete G1 application for deployment. **On-slide text:**

Wrapping It Up: Packaging the Application

- **Dependencies:** Ensuring all required libraries and models are included.
- **Configuration Files:** Bundling necessary settings (e.g., audio params, stance configs).
- **Launch Scripts:** Creating scripts to start all subsystems in the correct order.

Speaker notes: Once our application is built, we need to package it. This involves gathering all dependencies, bundling configuration files, and creating launch scripts that start all the necessary subsystems in the correct sequence. A well-packaged application is easy to deploy and run.

Diagram cue: A graphic showing various components (code, configs, models) being placed into a single "Application Package" box.

Slide 23: Deploying a Full Multimodal Application

Section: Unified Application Assembly **Purpose:** Explain the process of deploying the packaged application to G1. **On-slide text:**

Going Live: Deploying to G1

- **Transfer:** Moving the package to G1's onboard computer.
- **Environment Setup:** Configuring the runtime environment.
- **Execution:** Running the launch scripts and monitoring startup.

Speaker notes: Deploying the application involves transferring the package to G1, setting up the runtime environment, and executing the launch scripts. We'll discuss best practices for this process to ensure a smooth transition from development to live operation.

Diagram cue: Diagram showing the transfer of an application package from a developer's computer to G1's onboard computer.

Slide 24: Final Validation & Deployment - Introduction

Section: Final Validation & Deployment **Purpose:** Introduce the final testing and deployment phase. **On-slide text:**

The Final Check: Validation and Deployment

- **Full-Stack Testing:** Verifying the entire integrated system.
- **Deployment Habits:** Best practices for reliable operation.
- **Course Wrap-Up:** Reviewing our journey.

Speaker notes: Before we consider our application complete, we must perform rigorous final validation. This involves full-stack testing of the integrated system. We'll also discuss essential deployment habits for reliable operation and then wrap up our comprehensive G1 course.

Diagram cue: A checklist icon with a final checkmark being placed.

Slide 25: Full-Stack Testing

Section: Final Validation & Deployment **Purpose:** Detail the process of testing the complete, integrated application. **On-slide text:**

End-to-End: Full-Stack Testing

- **Scenario Testing:** Running through complete use cases (e.g., “Walk to table, grab cup, say ‘Done’”).
- **Edge Cases:** Testing unusual inputs or unexpected situations.
- **Performance Monitoring:** Checking CPU/memory usage and latency during operation.

Speaker notes: Full-stack testing is end-to-end validation. We run through complete scenarios, test edge cases to ensure robustness, and monitor system performance to guarantee our application runs smoothly and reliably under various conditions.

Diagram cue: A flowchart showing a complete scenario test, from user input to final G1 action and feedback.

Slide 26: Deployment Habits for Reliable Operation

Section: Final Validation & Deployment **Purpose:** Share best practices for deploying and maintaining G1 applications. **On-slide text:**

Built to Last: Deployment Habits

- **Version Control:** Track changes to your application code and configurations.
- **Automated Logging:** Ensure comprehensive logs are generated for debugging.
- **Regular Maintenance:** Schedule checks for hardware and software updates.

Speaker notes: Reliable operation requires good deployment habits. Using version control, ensuring automated logging, and scheduling regular maintenance are crucial practices that will save you time and headaches in the long run.

Diagram cue: Icons representing version control (e.g., Git), logging, and maintenance tools.

Slide 27: Course Wrap-Up - The G1 Journey

Section: Final Validation & Deployment **Purpose:** Review the key topics covered throughout the course. **On-slide text:**

Our G1 Journey: A Look Back

- **Days 1-4:** Fundamentals, hardware, basic control.
- **Day 5:** Communication (DDS) and Safety Workflows.

- **Day 6:** Locomotive and Arm Control Infrastructure.
- **Day 7:** AI Locomotion, Audio, and Application Assembly.

Speaker notes: Let's take a moment to look back at our journey. We started with the fundamentals, moved through communication and safety, tackled complex control infrastructures, and finally, assembled a fully integrated, intelligent application. You've covered a lot of ground!

Diagram cue: A timeline graphic showing the progression of topics from Day 1 to Day 7.

Slide 28: The Future of G1 Development

Section: Final Validation & Deployment **Purpose:** Inspire students to continue exploring and developing for G1. **On-slide text:**

Beyond the Course: The Future of G1

- **Advanced AI:** Exploring deeper reinforcement learning and computer vision.
- **Complex Manipulation:** Tackling intricate tasks and tool use.
- **Swarm Robotics:** Coordinating multiple G1 units.

Speaker notes: This course is just the beginning. The future of G1 development is vast, encompassing advanced AI, complex manipulation, and even swarm robotics. I encourage you to continue exploring, experimenting, and pushing the boundaries of what G1 can do.

Diagram cue: An inspiring image of a G1 humanoid in a futuristic setting or performing a complex task.

Slide 29: Day 7 Outcome Coverage Report

Section: Final Validation & Deployment **Purpose:** Summarize the coverage of required Day 7 outcomes. **On-slide text:**

Comprehensive Coverage: Day 7 Outcomes Achieved

Outcome	Coverage (%)	Notes
AI-Driven Locomotion & Validation	100%	Interfacing, data flow, state validation benchmarks (static/dynamic).
G1 Audio Infrastructure	100%	Parameter classes, service constants, playback routines.
Multimodal Speech Interfaces	100%	ASR, TTS, WAV management, LED synchronization.
Complete Application Assembly	100%	Blending modalities, packaging, deployment, full-stack testing.

Speaker notes: We have thoroughly covered all required Day 7 outcomes, ensuring a complete understanding of integrating AI, audio, and speech into a unified G1 application. Each topic has been addressed with practical workflows and beginner-friendly explanations.

Diagram cue: A pie chart or bar graph showing 100% coverage for each outcome.

Slide 30: Copyright and Disclaimer

Section: Copyright **Purpose:** State copyright information and disclaimers. **On-slide text:**

© 2026 Vinci AI. All rights reserved.

Property of Vinci AI — Do Not Distribute

Disclaimer: This material is for educational purposes only and should not be used for actual robot operation without proper training and supervision.

Speaker notes: This concludes our Day 7 session and the G1 course. Please remember that this material is the property of Vinci AI and is intended for educational use only. Always prioritize safety and follow proper protocols when working with robotics. Thank you for your participation!

Diagram cue: Vinci AI logo, subtle diagonal watermark, copyright tag.